An Introduction to Fractal Geometry, Illustrated by the Mandelbrot Set

New edition 2026

by

Urban Bettag

Facharbeit (advanced mathematics course)

Integrierte Gesamtschule Ernst Bloch

Ludwigshafen am Rhein

Preface

This *Facharbeit*, "An Introduction to Fractal Geometry, Illustrated by the Mandelbrot Set", was prompted by an article in the German-language edition of "Scientific American". That publication presented the background to an object of recreational mathematics in an accessible form and showed fascinating computer graphics.

The aim of this Facharbeit is to clarify the mathematical framework behind such complex structures. The structures themselves conceal deep mysteries, and their investigation is constantly advancing.

It is obvious that a Facharbeit can treat the subject only to a limited extent, since this is a relatively new branch of discrete mathematics and the mathematical relationships are more complicated than they are often made out to be.

Contents

Preface i

List of Figures iii

1 Introduction to complex numbers 1
- 1.1 General remarks . . . 1
- 1.2 Definition of the complex numbers . . . 1
- 1.3 The four basic arithmetic operations . . . 1
  - 1.3.1 Addition and subtraction . . . 1
  - 1.3.2 Multiplication . . . 1
  - 1.3.3 Division . . . 1
- 1.4 Basic laws of algebra in $\mathbb{C}$ . . . 2
- 1.5 The modulus (absolute value) in $\mathbb{C}$ . . . 2
- 1.6 Geometric interpretation . . . 3
  - 1.6.1 Addition and subtraction . . . 3
  - 1.6.2 Multiplication and division . . . 4
- 1.7 Ordering of complex numbers . . . 4
- 1.8 Examples . . . 5

2 Sequences and limits in $\mathbb{C}$ 6
- 2.1 Sequences and convergence in $\mathbb{C}$ . . . 6
  - 2.1.1 Definition of the limit . . . 6
- 2.2 Limit theorems . . . 6
- 2.3 Example . . . 7

3 The Mandelbrot Set 8
- 3.1 General Remarks . . . 8
- 3.2 Mathematical Description of the Mandelbrot Set . . . 8
- 3.3 The “Apfelmännchen” Algorithm . . . 9
- 3.4 Different Ways of Depicting Mandelbrot Sets . . . 11
- 3.5 Similar Structures within the Mandelbrot Set . . . 11
- 3.6 Further Findings about Mandelbrot Sets . . . 11

4 Fractal Dimension 13
- 4.1 The Koch Snowflake . . . 13
- 4.2 Fractal Dimension . . . 13

5 Appendix 14
- 5.1 Various images . . . 14
- 5.2 Afterword . . . 26
- 5.3 References . . . 27
- 5.4 Pascal listing . . . 28
- 5.5 Python post-processing from 2026 . . . 40

List of Figures

Complex number in Cartesian and polar coordinates 3
Addition and subtraction in the complex plane 3
Angles in the multiplication and division of complex numbers 4
Plate 1: Examples of iterated function systems 14
Plate 2: The Mandelbrot set in the complex plane 15
Plate 3: Effect of the iteration limit 16
Plate 4: Magnification of a boundary region 17
Plate 5: Coloured fractal images 18
Plate 6: Black-and-white recomputation 19
Plate 7: Another miniature copy 20
Plate 8: Two magnified details 21
Plate 9: Koch curve and scaling 22
Plate 10: Three-dimensional representation 23
Plate 11: Differentiated graphical representation 24
Plate 12: Detail with high iteration depth 25

Note on the new edition 2026

This edition was produced in 2026 on the basis of the 1990 Facharbeit. It follows the original's structure and line of argument, but has been reworded and considerably shortened: many calculations, examples, figures and individual sections of the original have been condensed or omitted, and mathematically incorrect statements have been corrected. Only some of these changes are marked individually in the highlighted note boxes and in the change log.

The 1990 original is preserved, together with a verbatim transcription and a facsimile of every page, in a private archive. The figures were recalculated or redrawn in 2026. The Python code used for this is documented in the appendix, separately from the original 1990 Pascal code.

1 Introduction to complex numbers

1.1 General remarks

In algebra, when solving equations, for example when finding the zeros of a function, one frequently encounters the problem that an equation has no real solution. Within the real numbers, the square root of a negative number cannot be taken. This restriction can be overcome by extending the number system and introducing a new number i with the property $i^2 = -1$.

1.2 Definition of the complex numbers

A complex number consists of a real component and an imaginary component. The set of complex numbers is defined by

$$\mathbb{C} = \{\, a + bi \mid a,b \in \mathbb{R} \,\}, \qquad i^2 = -1.$$

For $z = a + bi$, $a = \mathrm{Re}(z)$ is called the real part and $b = \mathrm{Im}(z)$ the imaginary part. The notation $i = \sqrt{-1}$ is to be understood only as an intuitive shorthand; the actual definition is $i^2 = -1$.

1.3 The four basic arithmetic operations

Let $z_1 = a + bi$ and $z_2 = c + di$ with $a,b,c,d \in \mathbb{R}$.

1.3.1 Addition and subtraction

$$z_1 + z_2 = (a + c) + i(b + d), \qquad z_1 - z_2 = (a - c) + i(b - d).$$

Complex numbers are added or subtracted by adding or subtracting their real parts and their imaginary parts separately.

1.3.2 Multiplication

$$z_1 z_2 = (a + bi)(c + di) = (ac - bd) + i(ad + bc).$$

Here the brackets are multiplied out as with real expressions, and $i^2 = -1$ is then substituted. For the complex conjugate $\overline{z} = a - bi$ we have, in particular,

$$z\overline{z} = (a + bi)(a - bi) = a^2 + b^2.$$

1.3.3 Division

For $z_2 \neq 0$, that is $c^2 + d^2 \neq 0$, the numerator and denominator of the fraction are multiplied by the complex conjugate of the denominator:

$$\frac{z_1}{z_2} = \frac{a + bi}{c + di} = \frac{(a + bi)(c - di)}{(c + di)(c - di)} = \frac{ac + bd}{c^2 + d^2} + i\,\frac{bc - ad}{c^2 + d^2}.$$

Both components of the result are real numbers.

1.4 Basic laws of algebra in $\mathbb{C}$

The arithmetic laws familiar from the real numbers also hold for complex numbers. For $z_1,z_2,z_3 \in \mathbb{C}$ we have the commutative law

$$z_1 + z_2 = z_2 + z_1, \qquad z_1 z_2 = z_2 z_1,$$

the associative law

$$(z_1 + z_2) + z_3 = z_1 + (z_2 + z_3), \qquad (z_1 z_2) z_3 = z_1 (z_2 z_3),$$

and the distributive law

$$z_1 (z_2 + z_3) = z_1 z_2 + z_1 z_3.$$

The identity elements are $0 = 0 + 0i$ for addition and $1 = 1 + 0i$ for multiplication. The additive inverse of $z = a + bi$ is $-z = -a - bi$. For $z \neq 0$ the multiplicative inverse also exists:

$$z^{-1} = \frac{\overline{z}}{z\overline{z}} = \frac{a - bi}{a^2 + b^2}.$$

The complex numbers with addition and multiplication therefore form a field.

> **Correction note** In the original, individual transformations were worked through in detail, but the identity elements and the multiplicative inverse were not stated in full. This gap, marked by the teacher, has been closed here.

1.5 The modulus (absolute value) in $\mathbb{C}$

The modulus of a complex number $z = a + bi$ is defined by

$$|z| = |a + bi| = \sqrt{a^2 + b^2}.$$

The following hold:

$$|z| \geq 0, \qquad |z| = |-z|, \qquad |z_1 z_2| = |z_1|\,|z_2|$$

and, for $z_2 \neq 0$,

$$\left|\frac{z_1}{z_2}\right| = \frac{|z_1|}{|z_2|}.$$

In addition, the triangle inequality holds:

$$|z_1 + z_2| \leq |z_1| + |z_2|.$$

To prove it, one can square both non-negative sides. With $z_1 = a + bi$ and $z_2 = c + di$ it then suffices to show

$$ac + bd \leq \sqrt{a^2 + b^2}\sqrt{c^2 + d^2},$$

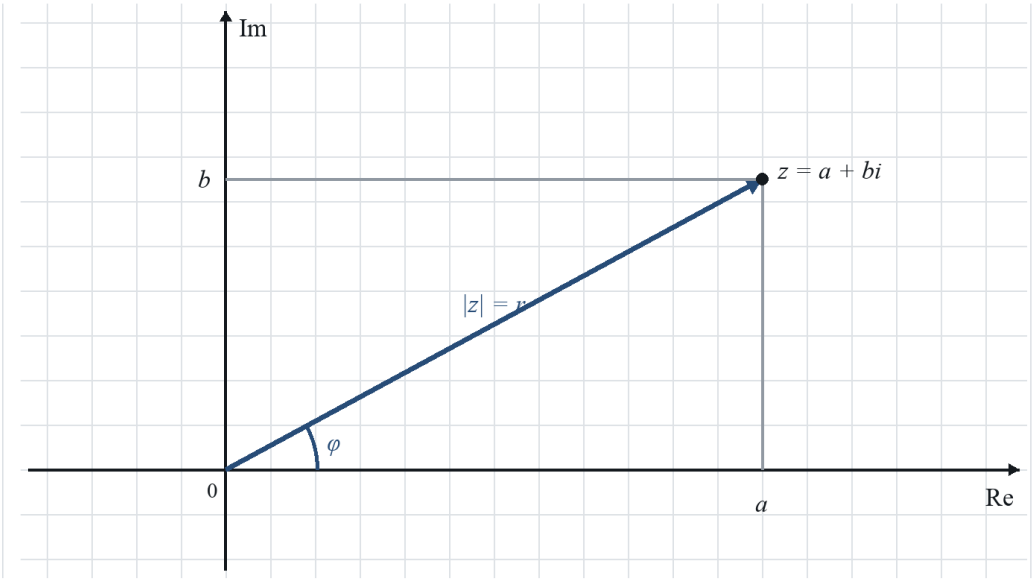
which is the Cauchy–Schwarz inequality in $\mathbb{R}^2$. Equivalently, the transformation used in the original ends with the true statement

$$0 \leq (ad - bc)^2.$$

In the complex plane (Gaussian plane), $z = a + bi$ corresponds to the point (a,b). The modulus $|z|$ is its distance from the origin, and $|z_1 - z_2|$ is the distance between the corresponding points. The conditions $|z| = 1$, $|z| < 1$ and $|z| > 1$ describe the unit circle, its interior and its exterior.

1.6 Geometric interpretation

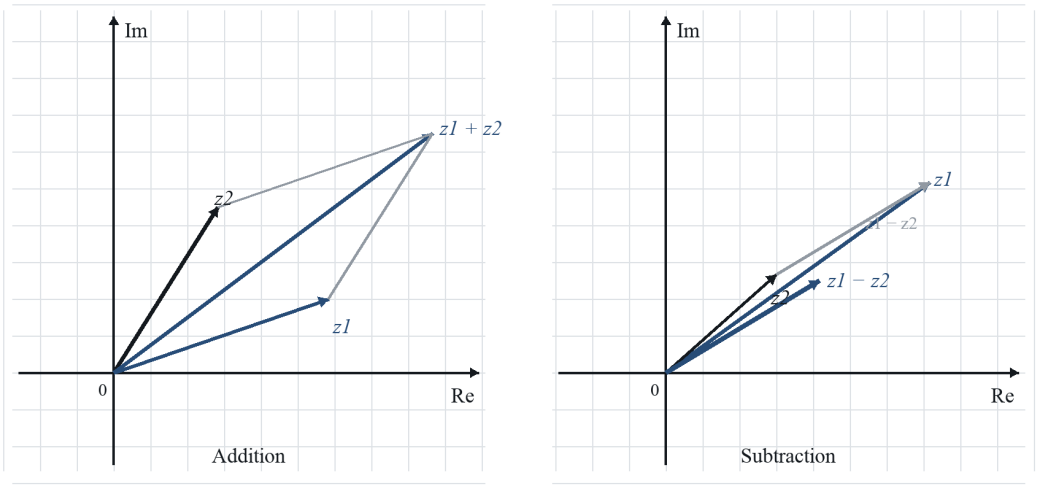
The real part is plotted on the horizontal axis of the complex plane and the imaginary part on the vertical axis. Addition and subtraction can thus be represented as vector addition and vector subtraction respectively.



The point $z = a + bi$ is determined by its two coordinates a and b. Equivalently, it can be described by its distance $r = |z|$ from the origin and the angle φ to the positive real axis.

1.6.1 Addition and subtraction

Every complex number can be regarded as a position vector. Completing the vectors to a parallelogram gives the sum of two complex numbers. The difference $z_1 - z_2$ is obtained by adding z_1 and $-z_2$.



When adding, $z_1 + z_2$ arises as the diagonal of the parallelogram. The difference $z_1 - z_2$ can be interpreted as the vector joining z_2 to z_1.

1.6.2 Multiplication and division

For $z \neq 0$ we introduce polar coordinates. With $r = |z| > 0$ and an argument φ we have

$$z = r(\cos\varphi + i\sin\varphi).$$

For

$$z_1 = r_1(\cos\varphi_1 + i\sin\varphi_1), \qquad z_2 = r_2(\cos\varphi_2 + i\sin\varphi_2)$$

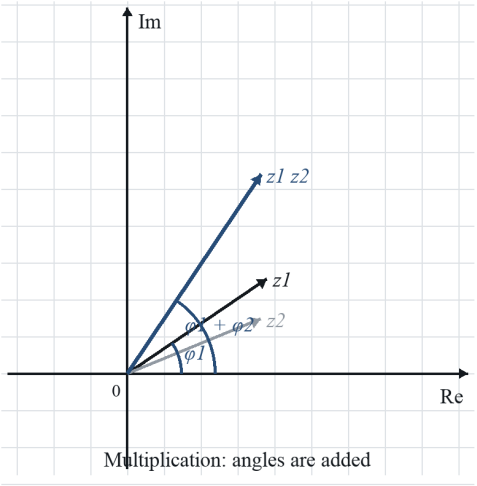
the angle-addition formulas give

$$z_1 z_2 = r_1 r_2\big(\cos(\varphi_1 + \varphi_2) + i\sin(\varphi_1 + \varphi_2)\big).$$

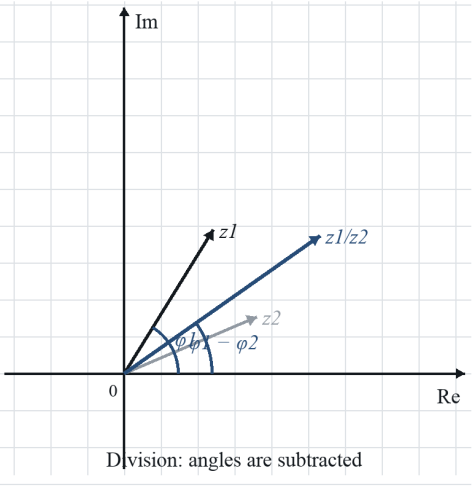
In multiplication, therefore, the moduli are multiplied and the arguments added. Correspondingly, for $z_2 \neq 0$,

$$\frac{z_1}{z_2} = \frac{r_1}{r_2}\big(\cos(\varphi_1 - \varphi_2) + i\sin(\varphi_1 - \varphi_2)\big).$$

In division, the moduli are divided and the arguments subtracted.



Multiplication: angles are added



Division: angles are subtracted

The drawings illustrate only the angle relationships. The length of the product vector or of the quotient vector also depends on the moduli $r_1 r_2$ or r_1/r_2 respectively.

1.7 Ordering of complex numbers

The complex numbers cannot be ordered in such a way that, together with the usual arithmetic operations, they form an ordered field. If $i > 0$, then compatibility with multiplication would give $i^2 = -1 > 0$. If, on the other hand, $i < 0$, then $-i > 0$, and likewise $(-i)^2 = -1 > 0$ would follow. Both contradict $1 > 0$.

As a mere set, $\mathbb{C}$ can of course be ordered, for instance lexicographically by real and imaginary part. Sorting by modulus and then by a fixed choice of argument is also possible. Such orderings, however, are not compatible with addition and multiplication in the way that the usual ordering of the real numbers is.

Correction note The original proof considered only the case $i > 0$, and the subsequent ordering by modulus and angle was not fully specified. Both of the teacher's remarks have been taken into account here.

1.8 Examples

Example 1 The equation $z^2 + 2 = 0$ has the two solutions

$$z_1 = i\sqrt{2}, \qquad z_2 = -i\sqrt{2}.$$

Example 2 For $z^4 + 3 = 0$, that is $z^4 = -3$, we write

$$-3 = 3\bigl(\cos(180^\circ) + i\sin(180^\circ)\bigr).$$

De Moivre's theorem gives the four solutions

$$z_k = \sqrt[4]{3}\left(\cos\frac{180^\circ + 360^\circ k}{4} + i\sin\frac{180^\circ + 360^\circ k}{4}\right), \qquad k = 0,1,2,3.$$

Their arguments are 45°, 135°, 225° and 315°.

Example 3 For $z^5 + 4 = 0$, that is $z^5 = -4$, we correspondingly have

$$z_k = \sqrt[5]{4}\left(\cos\frac{180^\circ + 360^\circ k}{5} + i\sin\frac{180^\circ + 360^\circ k}{5}\right), \qquad k = 0,1,2,3,4.$$

The five arguments are 36°, 108°, 180°, 252° and 324°. The value 342° given in the original was a slip of the pen.

2 Sequences and limits in $\mathbb{C}$

2.1 Sequences and convergence in $\mathbb{C}$

A complex sequence is a mapping

$$\mathbb{N} \longrightarrow \mathbb{C}, \qquad n \longmapsto z_n.$$

Writing $z_n = a_n + ib_n$, the real and imaginary parts each form a sequence of real numbers. A complex sequence is called convergent if the sequence of real parts and the sequence of imaginary parts both converge.

2.1.1 Definition of the limit

The sequence (z_n) converges to $z \in \mathbb{C}$ if for every $\varepsilon > 0$ there exists an $N \in \mathbb{N}$ such that

$$|z_n - z| < \varepsilon \qquad \text{for all } n \geq N.$$

One writes

$$\lim_{n\to\infty} z_n = z.$$

For $z_n = a_n + ib_n$ and $z = a + ib$,

$$z_n \longrightarrow z \quad \Longleftrightarrow \quad a_n \longrightarrow a \ \text{ and } \ b_n \longrightarrow b.$$

From $|z_n - z| < \varepsilon$ it follows immediately that $|a_n - a| < \varepsilon$ and $|b_n - b| < \varepsilon$. Conversely, one chooses the bound $\varepsilon/2$ for both real sequences; then

$$|z_n - z| \leq |a_n - a| + |b_n - b| < \varepsilon.$$

> **Correction note** In the original, only one direction of this equivalence was proved. The converse, which the teacher asked for, has been added. The definition of the limit must also be distinguished from the Cauchy criterion proper.

2.2 Limit theorems

Let $z_n \to z$ and $w_n \to w$. Then

$$z_n + w_n \longrightarrow z + w,$$
$$z_n - w_n \longrightarrow z - w,$$
$$z_n w_n \longrightarrow zw.$$

If $w \neq 0$, then w_n is also non-zero from some sufficiently large index onwards, and

$$\frac{z_n}{w_n} \longrightarrow \frac{z}{w}.$$

By the continuity of the modulus, it also follows that

$$|z_n| \longrightarrow |z|.$$

The statement about polar coordinates that followed in the original requires a restriction: if $z \neq 0$ and a continuous branch of the argument is chosen in a neighbourhood of z, then the correspondingly chosen arguments also converge. Without such a choice, angles are determined only up to multiples of 2π.

2.3 Example

Let

$$z_n = \frac{n^2 i^n}{n^3 + 1}.$$

Since $|i^n| = 1$,

$$|z_n| = \frac{n^2}{n^3 + 1} \leq \frac{1}{n} \longrightarrow 0.$$

Hence z_n converges to 0. For $\varepsilon > 0$ it suffices to choose some $N > 1/\varepsilon$, because then for all $n \geq N$

$$|z_n| < \varepsilon.$$

Geometrically, this means that from this index onwards all terms of the sequence lie inside the circle with centre 0 and radius ε.

> **Correction note** The original solution treated the numerator and denominator separately, even though their individual limits do not exist as finite numbers, and it reversed an ε-inequality incorrectly. The direct estimate of the modulus closes this gap without replacing the example.

3 The Mandelbrot Set

3.1 General Remarks

The Mandelbrot set, also known informally as the "Apfelmännchen" (literally "little apple man"), became widely known around 1980 through the work of Benoît B. Mandelbrot and is named after him. Mandelbrot worked at IBM's Thomas J. Watson Research Center in Yorktown Heights near New York. His book on the fractal geometry of nature made the new forms known beyond a small circle of specialists.

Mathematics increasingly concerned itself with complex feedback processes. Such processes have in principle long been known and, in simple cases, can be described by differential equations. Mandelbrot's work showed that iterated equations can also be used to investigate fragmented, irregular structures reminiscent of forms in nature, such as clouds or fern leaves. Computing the corresponding images became possible with fast computers.

An iteration uses the result of one computational step as the input of the next step. In general one writes

$$x_0 = k, \qquad x_{n+1} = f(x_n).$$

This gives

$$x_1 = f(k), \qquad x_2 = f(f(k)), \qquad x_3 = f(f(f(k))), \quad \ldots$$

The notation $f^2(x)$ here denotes the twofold application $f(f(x))$ and must not be confused with $(f(x))^2$. This procedure is called iteration or feedback.

3.2 Mathematical Description of the Mandelbrot Set

In this chapter we now attempt to present a complex function that opens up for us the world of aesthetic computer graphics. In principle, many complex functions can be iterated. For our example, the Apfelmännchen graphic, we choose the quadratic function

$$f_k(z) = z^2 + k, \qquad z = a + bi, \quad k = c + di, \quad a,b,c,d \in \mathbb{R}.$$

For the real and imaginary parts it follows that

$$f_k(a + bi) = (a^2 - b^2 + c) + i(2ab + d).$$

Every point of the complex plane is described by a complex number; its distance from the origin is the number's modulus. For a fixed constant k, the function is applied repeatedly, beginning with the starting value zero:

$$z_0 = 0, \qquad z_{n+1} = z_n^2 + k.$$

The first iterations are therefore

$$z_1 = k, \qquad z_2 = k^2 + k, \qquad z_3 = (k^2 + k)^2 + k, \quad \ldots$$

For the image construction that follows, the decisive question is whether this sequence remains bounded or eventually escapes to infinity.

Correction note Despite the starting value $z_0 = 0$, the original records the expression $k^2 + k$ as the first step. It is in fact the second term of the sequence, z_2. The calculation has therefore only been re-indexed; the original approach is retained.

Three Cases for the Special Case $k = 0$

The following case distinction from the original is correct for the special mapping $f_0(z) = z^2$, that is, for $k = 0$. Throughout, the moduli of the complex numbers are meant.

a) If $|z| < 1$, then $|z^{2^n}| = |z|^{2^n} \to 0$. The sequence approaches the origin.

b) If $|z| = 1$, all terms of the sequence have modulus 1. The terms themselves, however, need not be equal.

c) If $|z| > 1$, then $|z|^{2^n}$ grows without bound; the sequence escapes to infinity.

For the Mandelbrot set, however, it is $f_k(z) = z^2 + k$ that is investigated. Here the summand k must not be omitted. Membership therefore cannot be decided by the three cases at the unit circle alone. A simple counterexample is $k = -1$:

$$0 \longrightarrow -1 \longrightarrow 0 \longrightarrow -1 \longrightarrow \cdots$$

The orbit remains bounded, although it does not converge to zero.

Teacher's remark The red question "Wieso entfällt das k?" ("Why does the k disappear?") identifies precisely the gap in the original transition. The case distinction is therefore not deleted but explicitly retained as the special case $k = 0$.

3.3 The "Apfelmännchen" Algorithm

Sequences that tend towards a fixed value or towards a periodic sequence are attracted by an attractor. If, on the other hand, the moduli escape without bound, one can speak intuitively of the attractor infinity. For the Mandelbrot set, however, it is not required that the sequence converge to zero. What matters is whether the orbit starting at zero remains bounded:

$$\mathcal{M} = \left\{ k \in \mathbb{C} \;\middle|\; z_0 = 0, \quad z_{n+1} = z_n^2 + k, \quad (z_n) \text{ is bounded} \right\}.$$

Calculating individual sequences by hand can show their behaviour only for a few examples. To investigate a larger region of the complex plane, the repeated calculation is transferred to the computer. Essentially three loops are needed for this: two loops run through the pixels, and the third carries out the iteration for each pixel.

Correction note Only the incorrect membership criterion has been replaced. The original defined the Mandelbrot set via the convergence $f^n(z) \to 0$. The correct criterion is the boundedness of the critical orbit; the point under investigation is the parameter k, not an arbitrary starting value.

Carrying Out the Algorithm

As an example, we choose the region $-2 \le c \le 1$ and $-1.5 \le d \le 1.5$ of the complex plane. Each pixel is assigned a constant $k = c + di$. During the calculation, the variables a and b hold

the real and imaginary parts of the current term of the sequence.

1. From the image size and the chosen region, determine the step sizes for c and d.
2. Assign the parameter $k = c + di$ to the current pixel.
3. Set $a := 0$, $b := 0$ and the iteration counter $h := N$.
4. Compute
$$\mathrm{Re} := a^2 - b^2 + c, \qquad \mathrm{Im} := 2ab + d.$$
5. Set $a :=$ Re, $b :=$ Im and $h := h - 1$.
6. Repeat the calculation until $a^2 + b^2 > 4$ or $h = 0$.
7. If $h = 0$, the pixel is marked as not having escaped within the iteration limit. Otherwise it is certainly not a point of the Mandelbrot set.

The image region produced in this way is an approximation. With a finite number of iterations, a coloured point does not mean that membership has been proved, but only that the sequence has not escaped up to that point.

> **Correction note** In the original scheme the real and imaginary parts are computed but not assigned back to the variables a and b. Without this update no genuine iteration takes place. The two assignments have been added; the surviving Pascal listing also performs them.

Escape Radius and Julia Sets

Depending on the computing speed, the maximum number of iterations can be chosen, for example, between 25 and 150. A larger number improves the approximation, especially at the boundary, but increases the computing time.

For the critical orbit, the radius 2 suffices as the escape radius. If $|k| > 2$, then $z_1 = k$ already lies outside this circle. For $|k| \le 2$ and a later orbit point with $|z_n| > 2$ it follows that

$$|z_{n+1}| = |z_n^2 + k| \ge |z_n|^2 - |k| \ge |z_n|^2 - 2 > |z_n|.$$

The modulus then continues to grow and the orbit escapes to infinity. The equality $|z_n| = 2$ on its own, however, is not a uniform special case. For $k = -2$ the critical orbit reaches the value 2 and stays there; no general statement follows from this for all parameters or orbit points with modulus 2.

For comparison, we now consider the Julia sets. Mandelbrot and Julia sets arise from the same family of functions, but through two different questions:

- For the Mandelbrot set, the parameter k is varied while the starting value $z_0 = 0$ remains fixed.
- For a filled Julia set K_k, k is held fixed and the starting point z_0 is varied.

The two are linked by the important relationship

$$k \in \mathcal{M} \quad \Longleftrightarrow \quad K_k \text{ is connected.}$$

Here “connected” does not mean that any two points can necessarily be joined by a polygonal path within the set. That property would be stronger than the connectedness required here.

> **Historical context** The separation of the parameter plane and the dynamical plane, and the relationship between $k \in \mathcal{M}$ and the connectedness of K_k, were part of the theory in 1990. They are historical corrections and not modern standards applied after the fact.

3.4 Different Ways of Depicting Mandelbrot Sets

Mandelbrot sets can be depicted in many ways. Depending on the chosen configuration, for instance a colour or a black-and-white display, different effects can be achieved. If the iteration process is stopped as soon as the modulus exceeds the given escape radius, the number of iterations needed up to that point can be assigned a colour or a grey level. This yields, among others, the following possibilities:

a) Only those points whose orbit does not escape within the iteration limit are marked.

b) Points with the same escape time receive the same colour. This produces coloured bands reminiscent of contour lines.

c) Combining the two methods gives a differentiated depiction with many gradations.

d) The escape speed can be interpreted as height. This produces a depiction that appears three-dimensional, for example a “Mandelbrot lake” with surrounding plateaus.

e) Assigning a different colour to each iteration step produces particularly colourful images.

f) Replacing colours with tones allows a musical composition to be created from the iteration data.

Many other forms of depiction, and variations on them, are also possible.

3.5 Similar Structures within the Mandelbrot Set

A further aspect directly connected with the Mandelbrot set is its approximate self-similarity or scale invariance. Objects are called self-similar if, when magnified, they show similar structures at different scales.

If one examines small regions at the boundary of the Mandelbrot set and magnifies them, ever new details appear. Miniatures of the Apfelmännchen shape occur that are related to the main figure. The Mandelbrot set, however, is not exactly self-similar everywhere; its repetitions are distorted and surrounded by new structures.

Such self-similar or approximately self-similar patterns are also found in nature, for instance in the branching of trees, ferns or river systems.

3.6 Further Findings about Mandelbrot Sets

The quadratic function investigated in this paper is the simplest case. Polynomials of higher degree can also be iterated and produce their own parameter and dynamical sets. For polynomials

of degree three, several critical points must in general be taken into account. Additional complex parameters can lead to higher-dimensional parameter spaces; a single complex variable, by contrast, remains representable in the two-dimensional complex plane.

Many depictions possess symmetries. Other structures show a sensitive transition between order and chaotic behaviour. Chaos theory investigates how very complicated behaviour can arise from simple deterministic rules.

Fractal geometry is not only a subject of theoretical research. Its basic ideas can also be conveyed in teaching and education, provided that a distinction is drawn between illustrative images, numerical approximations and mathematically proven statements.

4 Fractal Dimension

4.1 The Koch Snowflake

Classical Euclidean geometry essentially describes smooth shapes such as lines, circles, rectangles, spheres and pyramids. Many shapes in nature, however, are more complicated. A well-known example of a fractal curve is the Koch snowflake.

One starts with a line segment as the initiator. This is divided into three parts of equal length; the middle third is replaced by two sides of an equilateral triangle. The resulting polygonal line is called the generator. If each of its four segments is again replaced according to the same rule, and this process is continued indefinitely, the result is the Koch curve.

In the process the boundary becomes ever longer, while the resulting figure remains within a bounded region. The curve does not have the ordinary integer dimension of a line. It is more intricately branched than a line, yet it does not fill an area. For such fractured shapes Benoît Mandelbrot introduced the term “fractal”.

4.2 Fractal Dimension

Suppose that each stage of an exactly self-similar figure consists of N reduced copies of the whole, each copy being scaled down by the factor $s > 1$. Then

$$N = s^D \qquad \Longrightarrow \qquad D = \frac{\log N}{\log s}.$$

This number D is called the similarity dimension. For a line segment, a square and a cube with the scaling factor $s = 3$, one obtains $N = 3$, $N = 9$ and $N = 27$, and hence the dimensions 1, 2 and 3.

For the Koch curve, each segment gives rise to four new segments, each scaled down by the factor 3. Therefore

$$D = \frac{\log 4}{\log 3} \approx 1.26186.$$

The dimension is thus not an integer, but lies between the dimension of a line and the dimension of a surface.

For sets that are not exactly self-similar, the formula above cannot be applied directly. A common numerical approximation is the box-counting method: one covers the set with squares of side length ε, counts the number $N(\varepsilon)$ of squares needed and examines the ratio

$$\frac{\log N(\varepsilon)}{\log(1/\varepsilon)} \quad \text{as} \quad \varepsilon \to 0.$$

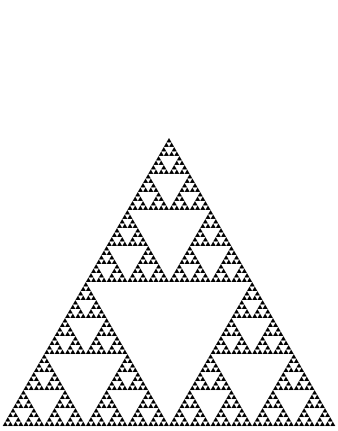
> **Historical context** The original left it unclear whether the dimension referred to the whole Mandelbrot set or to its boundary. The Mandelbrot set itself has interior points and therefore has Hausdorff dimension 2. That its boundary also has Hausdorff dimension 2 was proved only after 1990 and must not be treated retrospectively as having been known at the time.

5 Appendix

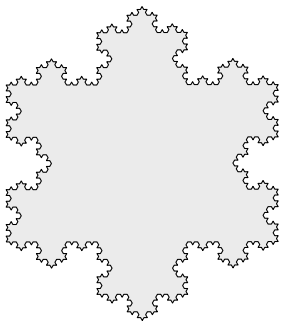
5.1 Various images

The following plates preserve the original selection of images and, where the original gives them, the coordinates and iteration depths. The Mandelbrot images with coordinates were recomputed in black and white with a Python program. The three-dimensional representation was likewise regenerated from the escape times. The colour plate was replaced by similar details, and the Koch curve and the scaling table were redrawn in LaTeX. The plate of IFS fractals shows three newly generated examples. The explanations beneath them make the mathematical meaning more precise.

Plate 1: Examples of iterated function systems (original page 31)



Sierpiński triangle



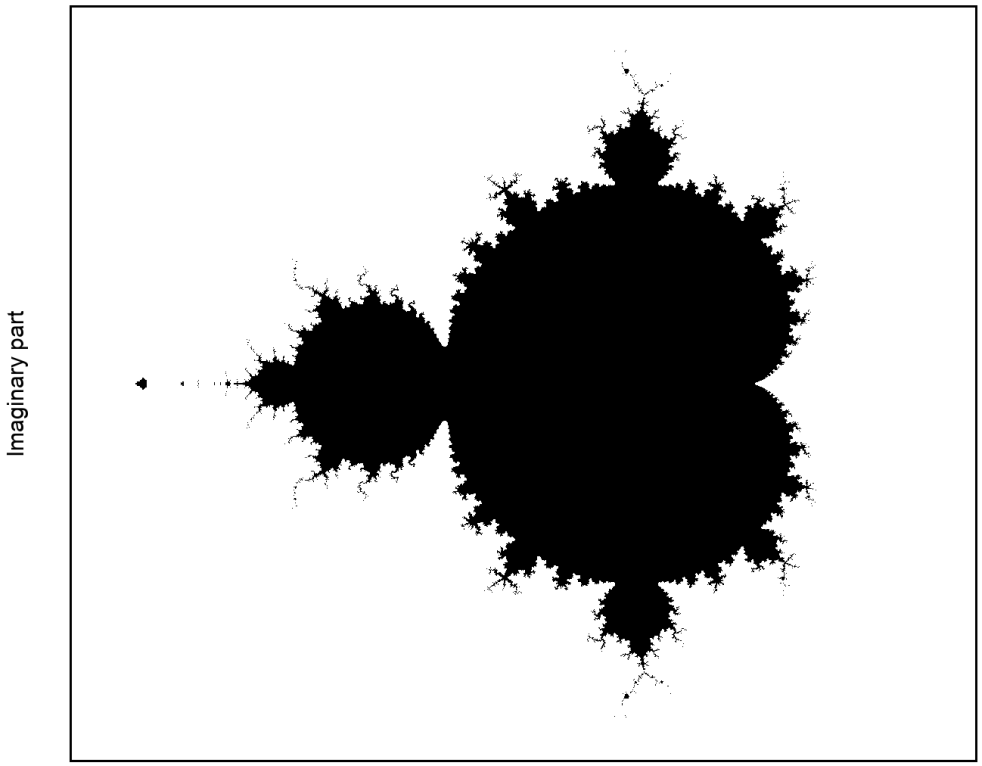
Koch snowflake



Barnsley fern

The original shows various IFS fractals, among them the Koch snowflake and the Sierpiński triangle. The remaining motifs lack labels or source references. The images were regenerated in 2026 and are restricted to examples with a known rule: the Sierpiński triangle and the snowflake are constructed exactly, and the fern is produced by the four affine maps given by Michael Barnsley (*Fractals Everywhere*, 1988). These figures are produced by iterated function systems. They do not belong to the Mandelbrot set, but they likewise show how complicated, self-similar structures can arise from a simple, repeated rule.

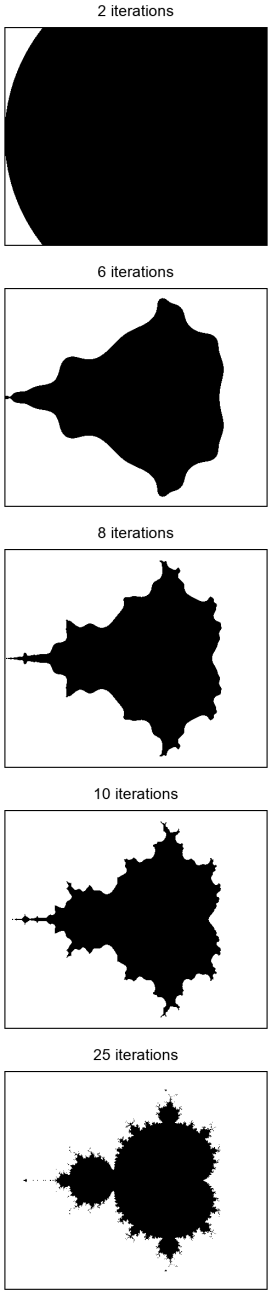
Plate 2: The Mandelbrot set in the complex plane (original page 32)




```
   Real part:  Minimum -2.00000000   Maximum 1.00000000
Imaginary part:  Minimum -1.25000000   Maximum 1.25000000
                   Iteration depth: 25
```

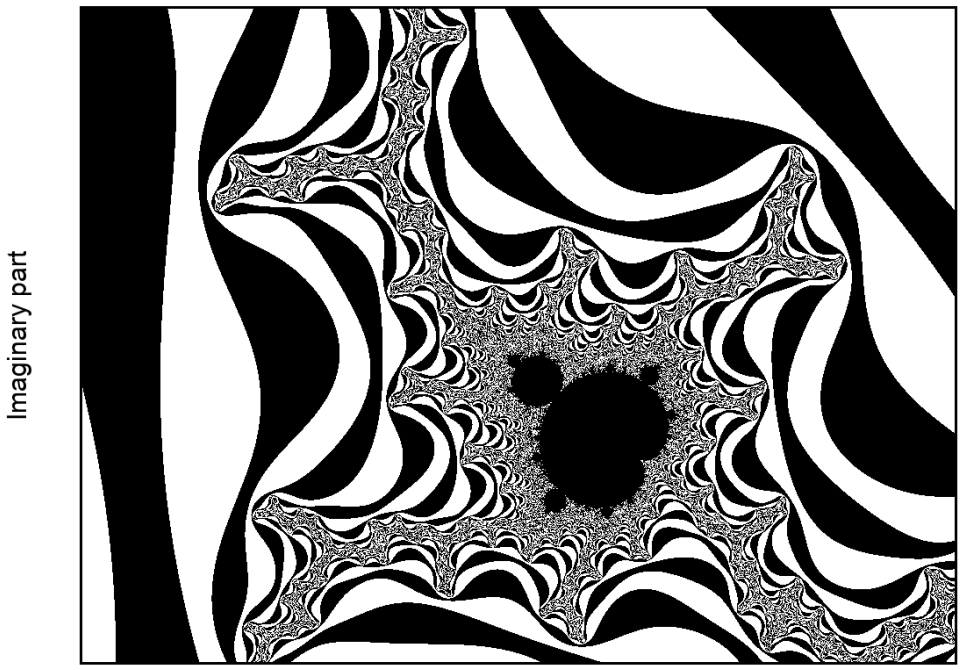
The black points have not escaped after 25 iterations. The picture is therefore a numerical approximation of the Mandelbrot set. A black point need not ultimately belong to the Mandelbrot set; its sequence may cross the escape boundary only after more than 25 steps. Convergence to zero is not assumed.

Plate 3: Effect of the iteration limit (original page 33)



The sequence of images shows the approximation after 2, 6, 8, 10 and 25 iterations. As the iteration limit rises, more and more points lying outside are recognised as escaping. The Mandelbrot set itself does not change; only the outer approximation becomes more accurate. Ever finer details therefore become visible, especially at the boundary, as the iteration depth increases.

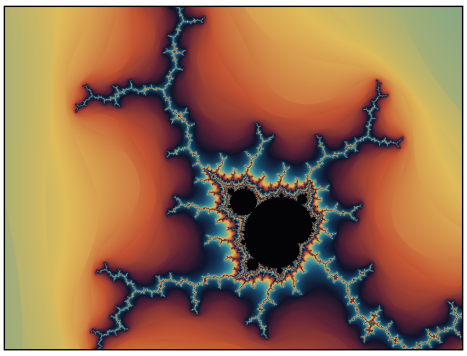
Plate 4: Magnification of a boundary region (original page 34)



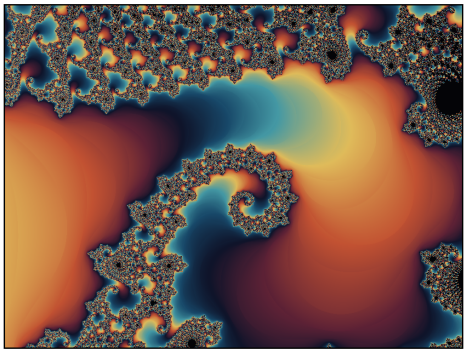

```
  Real part:  Minimum -0.19920000   Maximum -0.12954000
Imaginary part:  Minimum 1.01480000   Maximum 1.06707000
                 Iteration depth: 200
```

The image shows a magnification at the boundary of the Mandelbrot set. Further magnification brings out ever new structures and miniature copies. The region shown is only 0.06966 wide in the real part. The black and white bands denote different escape times; they are not additional parts of the Mandelbrot set.

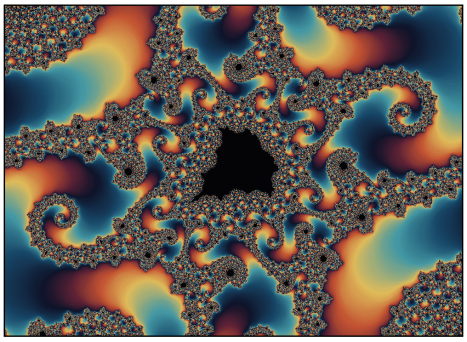
Plate 5: Coloured fractal images (original page 35)



Real part: Minimum −0.19920000 Maximum −0.12954000
Imaginary part: Minimum 1.01480000 Maximum 1.06707000
Iteration depth: 260



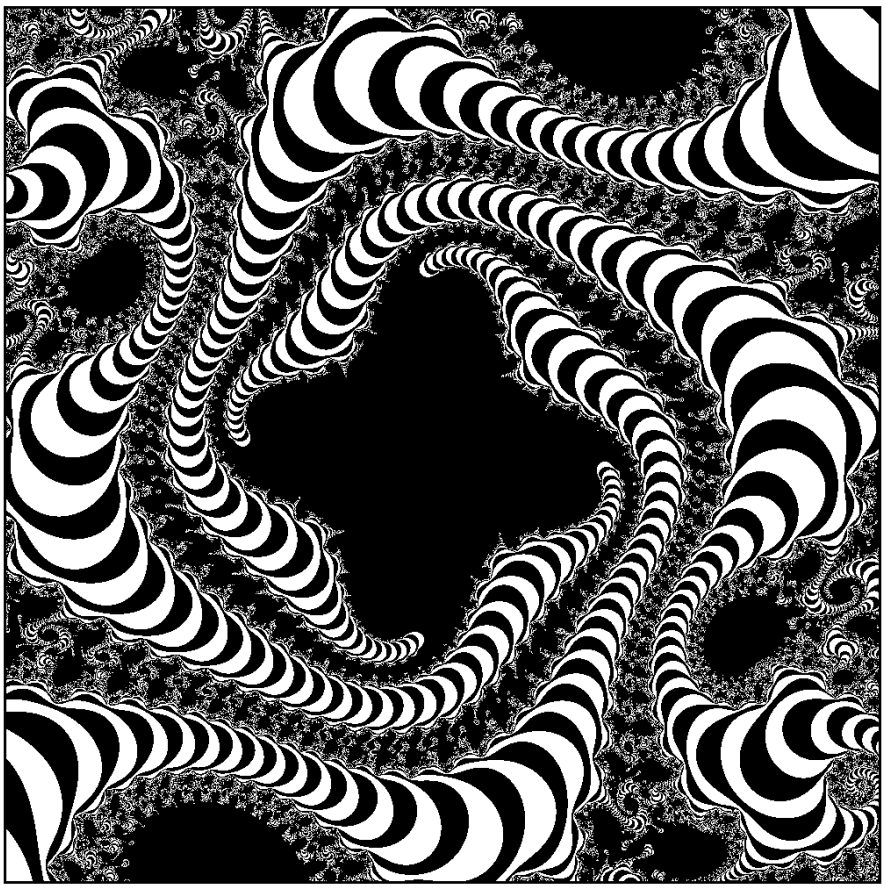
Real part: Minimum −0.74700000 Maximum −0.74000000
Imaginary part: Minimum 0.12900000 Maximum 0.13425000
Iteration depth: 320



Real part: Minimum −0.74543560 Maximum −0.74542150
Imaginary part: Minimum 0.11300370 Maximum 0.11301390
Iteration depth: 450

The original colour sheet survives in scan `IMG_1960` but gives no coordinates. For this reading edition, three similar details were therefore recomputed with a Python program; no claim is made to an exact reconstruction of the original images. The coordinate range used and the iteration depth are given beneath each image. The colours represent groups of similar escape times. They are not part of the Mandelbrot set as a mathematical set of points. A different colour assignment would therefore change the appearance, but not the computed sequences.

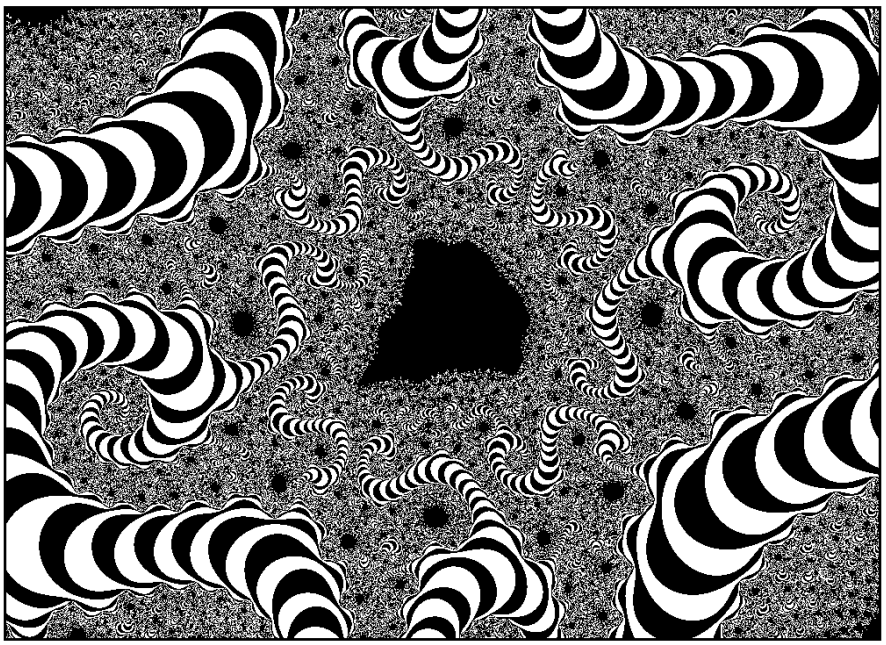
Plate 6: Black-and-white recomputation (original page 36)




```
   Real part:  Minimum -0.74638270   Maximum -0.74638210
Imaginary part:  Minimum -0.09850600   Maximum -0.09850540
                 Iteration depth: 350
```

A strong magnification with 350 iterations. The narrow black connections and recurring miniature copies illustrate the fine structure of the boundary. The width of the detail in the real part is only 0.00000060. The repetitions are similar to the main figure but not identical to it, since new structures appear at every level of magnification.

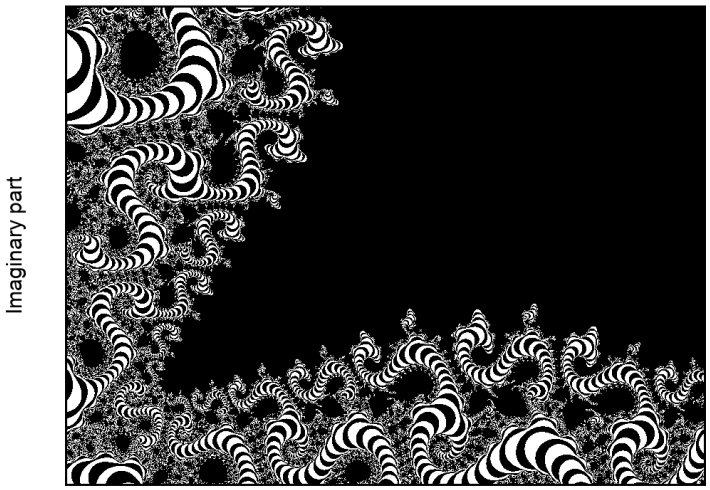
Plate 7: Another miniature copy (original page 37)



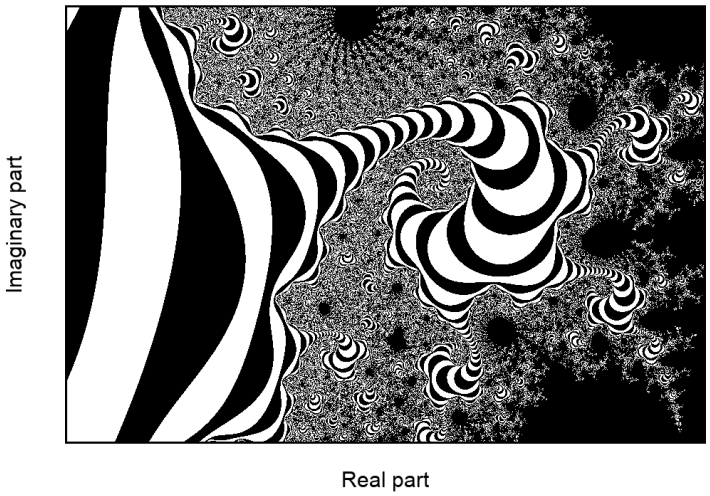

```
  Real part:  Minimum -0.74543560   Maximum -0.74542150
Imaginary part:  Minimum 0.11300370   Maximum 0.11301390
                 Iteration depth: 350
```

This detail, too, shows an approximate symmetry and a point-like miniature copy of the main figure. Its width in the real part is 0.00001410. The Mandelbrot set as a whole is exactly symmetric about the real axis. The local symmetry visible here, by contrast, is only approximate.

Plate 8: Two magnified details (original page 38)



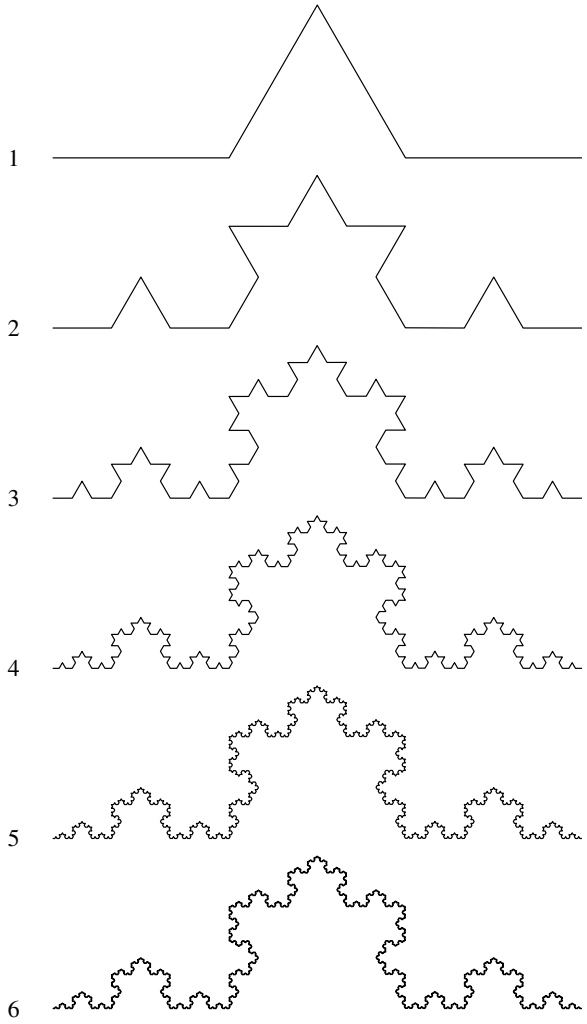

```
  Real part:  Minimum -0.74543010   Maximum -0.74542890
Imaginary part:  Minimum 0.11300760   Maximum 0.11300850
                 Iteration depth: 350
```

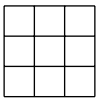
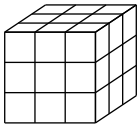



```
  Real part:  Minimum -0.75104000   Maximum -0.74080000
Imaginary part:  Minimum 0.10835000   Maximum 0.11536000
                 Iteration depth: 300
```

The images document different details and iteration depths of the computer representation of the time. The upper detail spans only 0.0000012 in the real part, the lower one 0.01024. The bands arise from showing the escape times alternately in black and white. They make the speed of escape visible, but they do not prove that an individual point belongs to the Mandelbrot set.

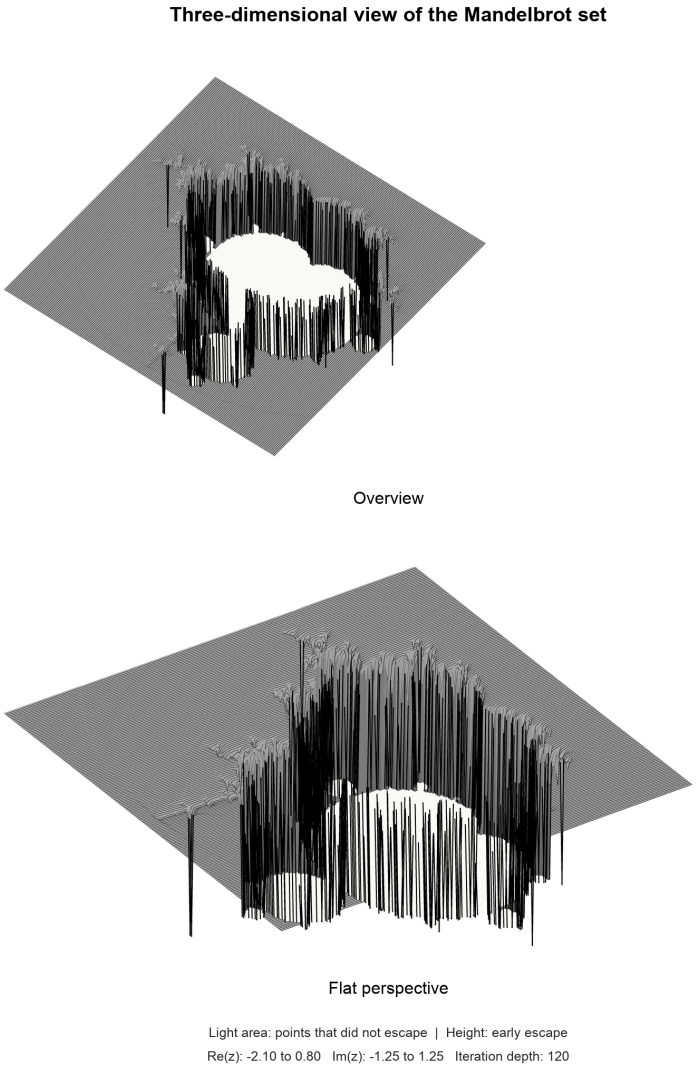
Plate 9: Koch curve and scaling (original page 39)



	Shape	Number N	Scaling s	Law
	Line segment	3	3	$3^1 = 3$
	Square	9	3	$3^2 = 9$
	Cube	27	3	$3^3 = 27$

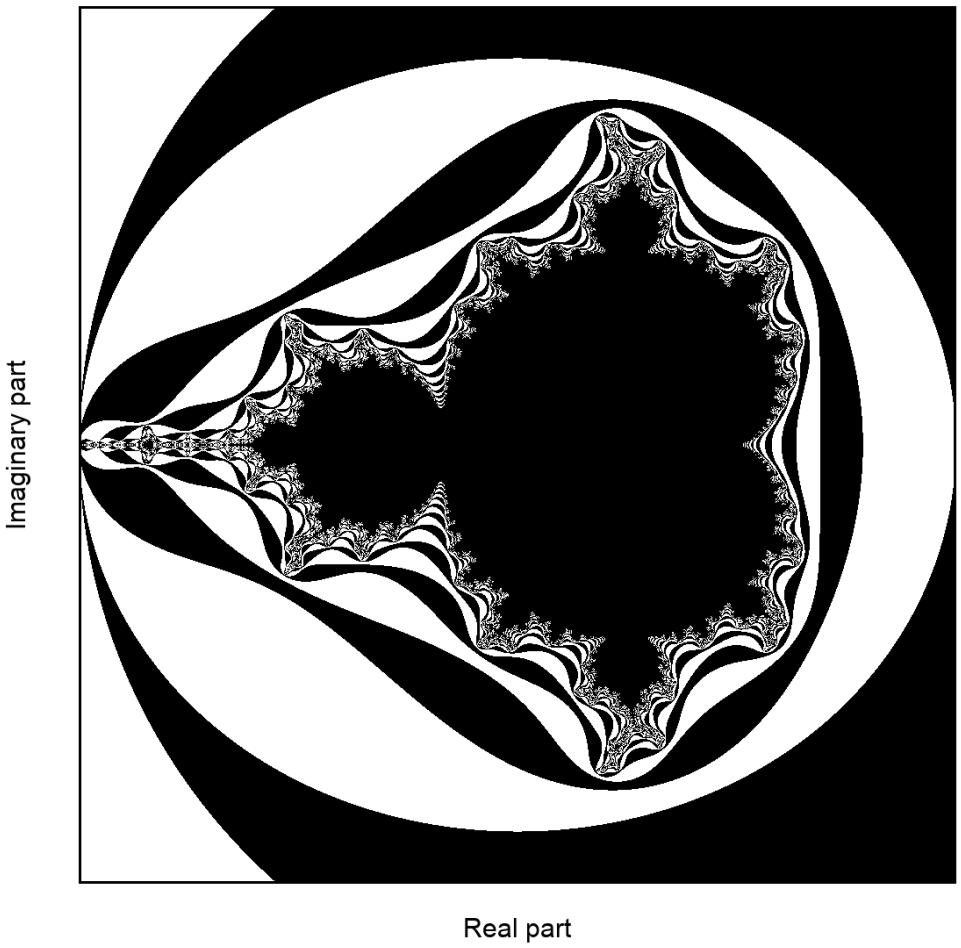
The iteration stages of the Koch curve and the table for line segment, square and cube illustrate the scaling law $N = s^D$. The curve and the table were redrawn from the original in 2026; the original's symbols a and d have been brought into line with the notation $N = s^D$ used in Section 4.2. The Koch curve is generated by a different rule from the Mandelbrot set. It serves here as a clear example on which self-similarity and a fractional dimension can be calculated directly.

Plate 10: Three-dimensional representation (original page 40)



The two perspectives were recomputed for $\mathrm{Re}(c) \in [-2.10, 0.80]$, $\mathrm{Im}(c) \in [-1.25, 1.25]$ and an iteration depth of 120; the original gives no coordinates for this plate, and the values were chosen in 2026. Points whose orbit does not escape within this limit form the light-coloured surface of the "Mandelbrot lake". For the remaining points, the height is chosen to be greater the earlier the orbit crosses the escape radius. The height is therefore not a third spatial coordinate of the Mandelbrot set but a visualisation of numerical data. Depending on the height function, the same computation can produce landscapes that look different; the planar Mandelbrot set remains unchanged.

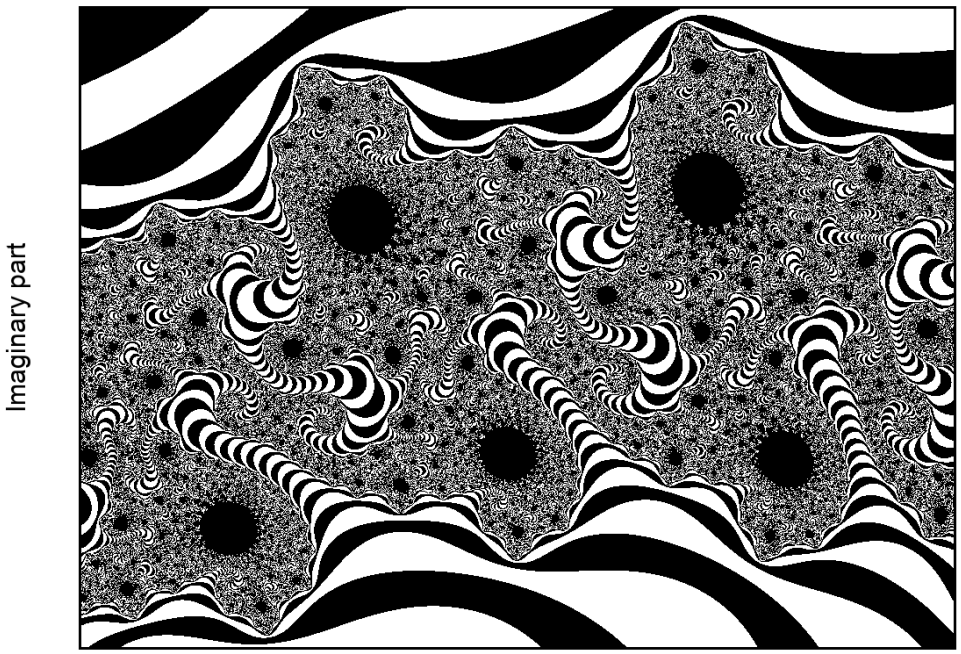
Plate 11: Differentiated graphical representation (original page 41)




```
   Real part:  Minimum -2.00000000   Maximum 1.00000000
Imaginary part:  Minimum -1.50000000   Maximum 1.50000000
                   Iteration depth: 25
```

The representation uses the differentiated colouring or shading explained in Section 3.4. Points with similar escape times are grouped into common bands. The outer rings and arcs thus show not only the boundary of the Mandelbrot set but also the behaviour of the escaping sequences in its neighbourhood.

Plate 12: Detail with high iteration depth (original page 42)




```
  Real part:  Minimum −0.74553800   Maximum −0.74505400
Imaginary part:  Minimum 0.11288100   Maximum 0.11323600
                   Iteration depth: 250
```

The detail was produced with an iteration depth of 250. Its width in the real part is only 0.000484. Despite this strong magnification, miniature copies and branching boundary structures appear once again. One can therefore speak of approximate self-similarity; an exact copy of the whole figure, however, does not occur.

5.2 Afterword

Are fractals merely mathematical games, or do they also prove their worth in practice? Fractal methods are in fact applied in various fields. Algorithms consisting of only a few instructions can generate artificial landscapes that are barely distinguishable from real ones. Such methods can be used in film production, for example, to supplement or replace elaborate models.

A further application lies in digital image transmission and image compression. Image information can be described compactly by suitable mathematical rules. This allows data to be transmitted faster, which matters particularly where storage space and transmission time are scarce or expensive.

The last chapters show that fractal geometry is an extensive field within mathematics. A Facharbeit can give only a general insight. The limits of school mathematical knowledge are quickly reached, since school mathematics is not sufficient for many of the details. The references list several works that deal with particular fractal structures and related topics.

5.3 References

[1] Benoît B. Mandelbrot. *The Fractal Geometry of Nature*. San Francisco: W. H. Freeman, 1982.

[2] Karl-Heinz Becker and Michael Dörfler. *Dynamische Systeme und Fraktale. Computergrafische Experimente mit Pascal*. 1st ed. Braunschweig: Vieweg, 1986.

[3] Heinz-Otto Peitgen and Dietmar Saupe, eds. *The Science of Fractal Images*. New York: Springer, 1988.

[4] Heinz-Otto Peitgen and Peter H. Richter. *The Beauty of Fractals. Images of Complex Dynamical Systems*. Berlin, Heidelberg, and New York: Springer, 1986.

[5] Hartmut Jürgens, Heinz-Otto Peitgen, and Dietmar Saupe. *Chaos und Fraktale*. Heidelberg: Spektrum der Wissenschaft Verlagsgesellschaft, 1989.

[6] Herbert Pieper. *Die komplexen Zahlen. Theorie – Praxis – Geschichte*. Deutsch-Taschenbücher 44. Thun and Frankfurt am Main: Harri Deutsch, 1985.

[7] Detlef Laugwitz. *Ingenieurmathematik I*. BI-Hochschultaschenbücher 59. Mannheim: Bibliographisches Institut, 1964.

[8] Heinz König. *Analysis I*. Basel, Boston, and Stuttgart: Birkhäuser, 1984.

[9] Immo Diener. *Computer-Kurzweil*. Heidelberg: Spektrum der Wissenschaft Verlagsgesellschaft, 1988.

> **Note on sources** The list follows the order and selection of the original scan. In 2026 the details were checked against library catalogues and publishers' information and supplemented with subtitles and series. Entry [3] has been corrected: *The Science of Fractal Images* is a volume edited by Peitgen and Saupe and appeared in 1988, not 1986. The original gives no page numbers for the individual statements and figures used.

5.4 Pascal listing

The printout of the program runs to eleven sheets: the first begins with `PROGRAM FRAKTAL;`, the last ends with `END.`. The following listing is a character-for-character transcription of this printout from 1990: comments, umlauts, spellings (including typing errors), brackets and indentation are reproduced as printed. Only lines that the printer had wrapped after 80 characters have been rejoined; the character hidden at the edge of the paper in the process was supplied from the context. The comments and on-screen texts in the listing are in German, as printed in 1990.

A complete compilation is nevertheless not possible, because the included files `GEMCONST.PAS`, `TRIXCONST.PAS`, `GEMTYPE.PAS`, `TRIXTYPE.PAS`, `GEMSUBS` and `TRIXSUBS` are not available. Routines such as `PAINT_RECT`, `GET_SCREEN` and `INIT_GEM` also come from them. The listing therefore documents the accessible source code, but without these contemporary libraries it is not a self-contained, compilable program.

Original Pascal source code from 1990

```
PROGRAM FRAKTAL;

 CONST

  {$I GEMCONST.PAS}         { Quelltext - Konstanten             }
  {$I TRIXCONST.PAS}        { Zusätzliche Quelltext - Konstanten }

 TYPE

  {$I GEMTYPE.PAS}          { Quelltext - Datentypen             }
  {$I TRIXTYPE.PAS}         { Zusätzliche Quelltext - Datentypen }

 VAR TASTE         : CHAR;           { einfacher Character                }
     I             : INTEGER;        { Wird im Zusammenhang mit Scan ver-
                                       wendet. Entspricht dem Zahlenwert
                                       der jeweiligen Taste                 }

     BILDSCHIRM    : SCREEN_TYPE;    { ST spezielles Bildschirm - Fomat    }
     BILD_SPEICHER : BOOLEAN;        { überprüft ob sich ein Bild im
                                       Speicher befindet. false entspricht
                                       einem leeren und true einem vollen
                                       Bildschirm-Speicher.                 }
     EN            : BOOLEAN;

  {$I GEMSUBS}              { Quelltext - Prozedur- und Funktionsköpfe }
  {$I TRIXSUBS}             { Weitere Prozedur- und Funktionsköpfe     }


 { Diese Routine fragt die Scancodes der Tastatur ab. Auf Grund dieser
   Routine ist es möglich auch spezielle Tasten (z.B. Funktionstasten)
   abzufragen. Dies ist mit dem herkömmlichen readln - Befehl nicht
   möglich.                                                          }

 FUNCTION BCONCIN(DEV : INTEGER) : LONG_INTEGER;BIOS(2);

 FUNCTION SCAN : INTEGER;

  BEGIN
   SCAN:=INT(BCONCIN (2) DIV (65535))
  END;

 { Prozedur zur Kursorsteuerung }

 PROCEDURE CURSOR(FUNKTION,RATE : INTEGER);XBIOS(21);
```

```
45
46
47  {-------------------------------------------------------------------------
48   FRACTALE AUSDRUCKEN
49   -------------------------------------------------------------------------}
50
51  PROCEDURE DRUCKEN;
52
53   CONST X_MAX = 639;
54         Y_MAX = 399;
55
56
57   VAR X,Y,I,J,CODE : INTEGER;
58       OK           : ARRAY[1..8] OF BOOLEAN;
59
60   BEGIN
61    IF (BILD_SPEICHER = FALSE) THEN
62     BEGIN
63      CUR_OFF;
64      CLEAR_SCREEN;
65      HIDE_MOUSE;
66      PAINT_STYLE(0);
67      PAINT_RECT(1,1,638,398);
68      PAINT_STYLE(1);
69      PAINT_RECT(4,4,632,392);
70      PAINT_STYLE(5);
71      PAINT_RECT(5,5,630,390);
72      PAINT_STYLE(1);
73      PAINT_RECT(232,189,194,34);
74      PAINT_STYLE(0);
75      PAINT_RECT(228,185,190,30);
76      GOTOXY(30,12);WRITELN('KEIN BILD IM SPEICHER');
77      I:=SCAN;
78     END
79    ELSE
80     BEGIN
81      PUT_SCREEN(BILDSCHIRM);
82      REWRITE(OUTPUT,'PRN:');    { Initialisierung des Druckers }
83      WRITELN(CHR(27),'U1');
84      WRITELN(CHR(27),'3',CHR(16));
85      X:=0;
86      Y:=0;
87      REPEAT
88       write(chr(27),'*',chr(6),chr((x_max+40+1) mod(256)),chr((x_max+40+1)div(256)));
89       for j:=1 to 40 do write(chr(0)); {linker Rand}
90       for x:=0 to x_max do
91        BEGIN
92         FOR I:=1 TO 8 DO OK[I]:=FALSE;
93         FOR I:=1 TO 8 DO
94          BEGIN
95           IF GET_PIXEL(X,Y+I-1)=BLACK THEN OK[I]:=TRUE;
96           IF I=8 THEN
97            BEGIN
98             CODE:=0;
99             IF OK[1]=TRUE THEN CODE:=CODE+128;
100            IF OK[2]=TRUE THEN CODE:=CODE+64;
101            IF OK[3]=TRUE THEN CODE:=CODE+32;
102            IF OK[4]=TRUE THEN CODE:=CODE+16;
103            IF OK[5]=TRUE THEN CODE:=CODE+8;
104            IF OK[6]=TRUE THEN CODE:=CODE+4;
105            IF OK[7]=TRUE THEN CODE:=CODE+2;
106            IF OK[8]=TRUE THEN CODE:=CODE+1;
107            WRITE(CHR(CODE));
108            IF X = X_MAX THEN FOR J:=1 TO (90*8-(X_MAX+1)-40) DO WRITE(CHR(0));
109           END;
110         END;
111       END;
```

```
      WRITELN(CHR(27),'J',CHR(8));
      Y:=Y+8;
    UNTIL Y>Y_MAX+8;
    REWRITE(OUTPUT,'CON:');
   END;
 END;

{-------------------------------------------------------------------------
 FRACTALE BERECHNEN
 -------------------------------------------------------------------------}

PROCEDURE BERECHNEN;

 VAR X_KOORDINATE,                  { x - Koordinate }
     Y_KOORDINATE,                  { y - Koordinate }
     RECHENTIEFE,                   { Rechentiefe    }
     ZAEHLER,                       { Zähler für Iterationen }
     X_Z,                           { Zähler für x - Koordinate }
     Y_Z           : INTEGER;       { Zähler für y - Koordinate }
     REAL_MIN,                      { Realteil - Minimum }
     REAL_MAX,                      { Realteil - Maximum }
     IMAG_MIN,                      { Imaginärteil - Minimum }
     IMAG_MAX,                      { Imaginärteil - Maximum }
     X_SCHRITT,                     { Schrittweite in x - Richtung }
     Y_SCHRITT,                     { schrittweite in y - Richtung }
     I_KONST,                       { Imaginärteil der Konstante }
     R_KONST,                       { Realteil der Konstante }
     REAL_ZAHL,                     { Neu errechneter Realteil }
     IMAG_ZAHL,                     { Neu errechneter Imaginärteil }
     REAL_Q,                        { Quadrierter Realteil }
     IMAG_Q       : REAL;           { Quadrierter Imaginäreil }
     STD,MIN,SEC  : INTEGER;        { Variabeln für Rechenzeit }


   {-------------------------------------------------------------------------
    GESTALTUNG DES HINTERGRUNDS UND DER EINGABEMASKE
    -------------------------------------------------------------------------}

   PROCEDURE HINTERGRUND;

    VAR A : INTEGER;

    BEGIN
     CUR_OFF;
     CLEAR_SCREEN;
     HIDE_MOUSE;
     PAINT_STYLE(0);
     PAINT_RECT(1,1,638,398);
     PAINT_STYLE(1);
     PAINT_RECT(4,4,632,392);
     PAINT_STYLE(5);
     PAINT_RECT(5,5,630,390);

     { Gestaltung des Eingabefensters }

     PAINT_STYLE(0);
     PAINT_RECT(113,50,413,65);
     PAINT_STYLE(1);
     PAINT_RECT(116,53,407,59);
     PAINT_STYLE(5);
     PAINT_RECT(117,54,405,57);
     PAINT_STYLE(1);
     PAINT_RECT(125,60,389,45);
     PAINT_STYLE(0);
     PAINT_RECT(123,58,389,45);
     TEXT_STYLE(0);
     TEXT_HEIGTH(13);
```

```
179       DRAW_STRING(235,85,'  Fractale berechnen  ');
180
181       { Gestaltung der Eingabebox }
182
183       PAINT_STYLE(0);
184       PAINT_RECT(113,120,413,246);
185       PAINT_STYLE(1);
186       PAINT_RECT(116,123,407,240);
187       PAINT_STYLE(5);
188       PAINT_RECT(117,124,405,238);
189       PAINT_STYLE(1);
190       PAINT_RECT(165,160,309,156);
191       PAINT_STYLE(0);
192       PAINT_RECT(163,158,309,156);
193       FOR A:=0 TO 3 DO
194        BEGIN
195         PAINT_STYLE(1);
196         PAINT_RECT(172,174+(A*32),143,25);
197         PAINT_STYLE(0);
198         PAINT_RECT(170,172+(A*32),143,25);
199         PAINT_STYLE(1);
200         PAINT_RECT(323,174+(A*32),140,25);
201         PAINT_STYLE(0);
202         PAINT_RECT(321,172+(A*32),140,25);
203        END;
204       GOTOXY(22,11);WRITELN('Realteil Minimum');
205       GOTOXY(22,13);WRITELN('Realteil Maximum');
206       GOTOXY(22,15);WRITELN('Imaginär Minimum');
207       GOTOXY(22,17);WRITELN('Imaginär Maximum');
208      END;
209
210
211  BEGIN
212
213   { Eingabe der Parameter und Fehlerüberprüfung }
214
215   REPEAT
216    HINTERGRUND;
217    CUR_ON;
218    CURSOR(3,0);
219    GOTOXY(42,11);READLN(REAL_MIN);
220    GOTOXY(42,13);READLN(REAL_MAX);
221    GOTOXY(42,15);READLN(IMAG_MIN);
222    GOTOXY(42,17);READLN(IMAG_MAX);
223    IF (REAL_MIN >= REAL_MAX) OR (IMAG_MIN >= IMAG_MAX) THEN
224     BEGIN
225      CUR_OFF;
226      PAINT_STYLE(1);
227      PAINT_RECT(165,160,309,156);
228      PAINT_STYLE(0);
229      PAINT_RECT(163,158,309,156);
230      DRAW_STRING(200,85,'E I N G A B E F E H L E R  !!!');
231      DRAW_STRING(210,200,'Achtung ! Das Minimum darf ');
232      DRAW_STRING(210,230,'nie größer als das Maximum ');
233      DRAW_STRING(210,260,'          sein !           ');
234      I:=SCAN;
235      CUR_ON;
236     END;
237   UNTIL (REAL_MAX > REAL_MIN) AND (IMAG_MAX > IMAG_MIN);
238
239   { Gestaltung des Hintergrunds }
240
241   CUR_OFF;
242   CLEAR_SCREEN;
243   HIDE_MOUSE;
244   PAINT_STYLE(0);
245   PAINT_RECT(1,1,638,398);
```

```
246  PAINT_STYLE(1);
247  PAINT_RECT(4,4,632,392);
248  PAINT_STYLE(5);
249  PAINT_RECT(5,5,630,390);
250
251  { Gestaltung der Eingabebox }
252
253  PAINT_STYLE(0);
254  PAINT_RECT(113,50,413,65);
255  PAINT_STYLE(1);
256  PAINT_RECT(116,53,407,59);
257  PAINT_STYLE(5);
258  PAINT_RECT(117,54,405,57);
259  PAINT_STYLE(1);
260  PAINT_RECT(125,60,389,45);
261  PAINT_STYLE(0);
262  PAINT_RECT(123,58,389,45);
263  TEXT_STYLE(0);
264  TEXT_HEIGTH(13);
265  DRAW_STRING(235,85,'  Fractale berechnen  ');
266  PAINT_STYLE(0);
267  PAINT_RECT(113,120,413,246);
268  PAINT_STYLE(1);
269  PAINT_RECT(116,123,407,240);
270  PAINT_STYLE(5);
271  PAINT_RECT(117,124,405,238);
272  PAINT_STYLE(1);
273  PAINT_RECT(165,160,309,156);
274  PAINT_STYLE(0);
275  PAINT_RECT(163,158,309,156);
276  CUR_ON;
277  GOTOXY(23,11);WRITELN('Bitte wählen Sie einen geeigneten');
278  GOTOXY(23,12);WRITELN('     Wert für die Anzahl der     ');
279  GOTOXY(23,13);WRITELN('          Iterationen !          ');
280  CUR_OFF;
281  TEXT_HEIGTH(4);
282  DRAW_STRING(175,230,'ACHTUNG !!!');
283  DRAW_STRING(175,238,'Wählen Sie für die Anzahl der Iterationen einen');
284  DRAW_STRING(175,246,'kleinen Wert, so ensteht ein grobes Bild ohne  ');
285  DRAW_STRING(175,254,'Feinstruktur. Bei gößeren Werten sind Fein-    ');
286  DRAW_STRING(175,262,'strukturen zu erkennen, die aber lange Rechen- ');
287  DRAW_STRING(175,270,'zeiten beanspruchen.(=> Werte zwischen 100 und ');
288  DRAW_STRING(175,278,'350 günstig )                                  ');
289
290  CUR_ON;
291  GOTOXY(37,18);WRITELN('>');
292  REPEAT
293   GOTOXY(38,18);READLN(RECHENTIEFE);
294  UNTIL RECHENTIEFE>0;
295  CUR_OFF;
296  HIDE_MOUSE;
297  CLEAR_SCREEN;
298
299  { Rechenzeit auf Null setzen }
300
301  STD:=0;MIN:=0;SEC:=0;
302  SETTIME(STD,MIN,SEC);
303
304  {------------------------------------------------------------------------
305   DER APFELMÄNNCHEN-ALGORITHMUS
306    ------------------------------------------------------------------------}
307
308  X_KOORDINATE:=400;
309  Y_KOORDINATE:=400;
310
311  { Berechnung der x- und y-Schrittweite }
312
```

```
 X_SCHRITT:=ABS(((REAL_MAX)-(REAL_MIN))/X_KOORDINATE);
 Y_SCHRITT:=ABS(((IMAG_MAX)-(IMAG_MIN))/Y_KOORDINATE);

 I_KONST:=IMAG_Min+Y_SCHRITT;
 FOR Y_Z:=0 TO Y_KOORDINATE DO
  BEGIN
   R_KONST:=REAL_MIN+X_SCHRITT;
   FOR X_Z:=0 TO X_KOORDINATE DO
    BEGIN
     REAL_ZAHL:=R_KONST;
     IMAG_ZAHL:=I_KONST;
     ZAEHLER:=RECHENTIEFE;
     REPEAT
      REAL_Q:=REAL_ZAHL*REAL_ZAHL;
      IMAG_Q:=IMAG_ZAHL*IMAG_ZAHL;
      IMAG_ZAHL:=2*REAL_ZAHL*IMAG_ZAHL+I_KONST;
      REAL_ZAHL:=REAL_Q-IMAG_Q+R_KONST;
      ZAEHLER:=ZAEHLER-1;
     UNTIL (REAL_Q+IMAG_Q>4) OR (ZAEHLER=0);
     IF ODD(ZAEHLER) OR (ZAEHLER=0)THEN PUT_PIXEL(X_Z,Y_Z,1);
     R_KONST:=R_KONST+X_SCHRITT;
    END;
  I_KONST:=I_KONST+Y_SCHRITT;
 END;

 { Parameterbeschriftung }

 GOTOXY(55,3 );WRITELN('Realteil:');
 GOTOXY(55,4 );WRITELN('Minimum: ',REAL_MIN:8:8);
 GOTOXY(55,5 );WRITELN('Maximum: ',REAL_MAX:8:8);
 GOTOXY(55,7 );WRITELN('Imaginärteil:');
 GOTOXY(55,8 );WRITELN('Minimum: ',IMAG_MIN:8:8);
 GOTOXY(55,9 );WRITELN('Maximum: ',IMAG_MAX:8:8);
 GOTOXY(55,11);WRITELN('Rechentiefe: ',RECHENTIEFE);
 GETTIME(STD,MIN,SEC);
 GOTOXY(55,13);WRITELN('Rechenzeit:  ',STD,':',MIN,':',SEC);
 GET_SCREEN(BILDSCHIRM); { Aktueller Bildschirminhalt wird im
                           Bildschirmspeicher abgelegt }
 BILD_SPEICHER:=TRUE;
 I:=SCAN;
 SHOW_MOUSE;
END;

{------------------------------------------------------------------------
 AUSGABE DER AKTUELLEN PARAMETER
 ------------------------------------------------------------------------}

PROCEDURE PARAMETER;

 BEGIN
  IF (BILD_SPEICHER = FALSE) THEN
   BEGIN
    CUR_OFF;
    CLEAR_SCREEN;
    HIDE_MOUSE;
    PAINT_STYLE(0);
    PAINT_RECT(1,1,638,398);
    PAINT_STYLE(1);
    PAINT_RECT(4,4,632,392);
    PAINT_STYLE(5);
    PAINT_RECT(5,5,630,390);
    PAINT_STYLE(1);
    PAINT_RECT(232,189,194,34);
    PAINT_STYLE(0);
    PAINT_RECT(228,185,190,30);
    GOTOXY(30,12);WRITELN('KEIN BILD IM SPEICHER');
   END
```

```
  ELSE
   PUT_SCREEN(BILDSCHIRM);
   I:=SCAN;
 END;

{-------------------------------------------------------------------------
 FRACTALE SPEICHERN UND LADEN
 -------------------------------------------------------------------------}

PROCEDURE IM_EXPORT;

 VAR FEHLER     : ERR_CODE;      { Fehler-Code beim Laden/Speichern }
     PFAD       : STRING;        { Pfad-Name der Datei              }
     NAME       : STRING;        { Name der Datei                   }
     D          : INTEGER;

BEGIN

 REPEAT

  { Gestaltung des Hintergrunds }

  CLEAR_SCREEN;
  PAINT_STYLE(0);
  PAINT_RECT(1,1,638,398);
  PAINT_STYLE(1);
  PAINT_RECT(4,4,632,392);
  PAINT_STYLE(5);
  PAINT_RECT(5,5,630,390);

  { Gestaltung der Eingabebox }

  PAINT_STYLE(0);
  PAINT_RECT(213,100,213,65);
  PAINT_STYLE(1);
  PAINT_RECT(216,103,207,59);
  PAINT_STYLE(5);
  PAINT_RECT(217,104,205,57);
  PAINT_STYLE(1);
  PAINT_RECT(225,110,189,45);
  PAINT_STYLE(0);
  PAINT_RECT(223,108,189,45);
  TEXT_STYLE(0);
  TEXT_HEIGTH(13);
  DRAW_STRING(235,125,'  Erstellte Bilder  ');
  DRAW_STRING(235,144,'Im- bzw. Exportieren');

  { Unterer Kasten in der Mitte }

  PAINT_STYLE(0);
  PAINT_RECT(213,170,213,150);
  PAINT_STYLE(1);
  PAINT_RECT(216,173,207,144);
  PAINT_STYLE(5);
  PAINT_RECT(217,174,205,142);
  FOR D:=0 TO 2 DO BEGIN
   PAINT_STYLE(0);
   PAINT_RECT(220,185+(D*42),40,39);
   PAINT_STYLE(1);
   PAINT_RECT(225,190+(D*42),32,31);
   PAINT_STYLE(0);
   PAINT_RECT(223,188+(D*42),32,31);
   PAINT_RECT(265,185+(D*42),154,39);
   PAINT_STYLE(1);
   PAINT_RECT(270,190+(D*42),146,31);
   PAINT_STYLE(0);
   PAINT_RECT(268,188+(D*42),146,31);
```

```
  END;

  { Beschriftung der Menüboxen }

  TEXT_STYLE(0);
  TEXT_HEIGTH(13);
  DRAW_STRING(227,208,'F 6');
  DRAW_STRING(227,250,'F 7');
  DRAW_STRING(227,292,'ESC');
  TEXT_HEIGTH(4);
  DRAW_STRING(278,200,'   Erstellte Bilder   ');
  DRAW_STRING(278,209,'importieren bzw. laden');
  DRAW_STRING(278,238,'   Erstellte Bilder   ');
  DRAW_STRING(278,246,'   exportieren bzw.   ');
  DRAW_STRING(278,254,'      speichern       ');
  DRAW_STRING(278,288,' Zurück ins Hauptmenü ');
  I:=SCAN;
  IF I=64 THEN
   BEGIN
    SHOW_MOUSE;
    NAME:='';
    PFAD:='A:\*.*';
    IF GET_IN_FILE(PFAD,NAME) THEN
     BEGIN
      CLEAR_SCREEN;
      FEHLER:=READ_SCREEN(1,NAME,BILDSCHIRM);
      PUT_SCREEN(BILDSCHIRM);
      BILD_SPEICHER:=TRUE;
      I:=SCAN;
     END;
   END;
  IF I=65 THEN
   BEGIN
    IF BILD_SPEICHER = FALSE THEN
     BEGIN
      CUR_OFF;
      CLEAR_SCREEN;
      HIDE_MOUSE;
      PAINT_STYLE(0);
      PAINT_RECT(1,1,638,398);
      PAINT_STYLE(1);
      PAINT_RECT(4,4,632,392);
      PAINT_STYLE(5);
      PAINT_RECT(5,5,630,390);
      PAINT_STYLE(1);
      PAINT_RECT(232,189,194,34);
      PAINT_STYLE(0);
      PAINT_RECT(228,185,190,30);
      GOTOXY(30,12);WRITELN('KEIN BILD IM SPEICHER');
      I:=SCAN;
     END
    ELSE
     BEGIN
      SHOW_MOUSE;
      CLEAR_SCREEN;
      PARAMETER;
      NAME:='';
      PFAD:='A:\*.*';
      IF GET_IN_FILE(PFAD,NAME) THEN
       BEGIN
        FEHLER:=WRITE_SCREEN(1,NAME,BILDSCHIRM);
       END;
     END;
   END;
 UNTIL I=1;
END;
```

```
{------------------------------------------------------------------------
 INFORMATION
 ------------------------------------------------------------------------}

PROCEDURE INFO;

 BEGIN
  CLEAR_SCREEN;
  PAINT_STYLE(0);
  PAINT_RECT(1,1,638,398);
  PAINT_STYLE(1);
  PAINT_RECT(4,4,632,392);
  PAINT_STYLE(5);
  PAINT_RECT(5,5,630,390);
  PAINT_STYLE(0);
  PAINT_RECT(50,40,540,320);
  PAINT_STYLE(1);
  PAINT_RECT(60,50,520,300);
  PAINT_STYLE(0);
  PAINT_RECT(58,48,520,300);
  TEXT_HEIGTH(13);
  TEXT_STYLE(0);
  DRAW_STRING(70,70,'Information');
  DRAW_STRING(70,110,'Das vorliegende Computerprogramm ermöglicht die graphische');
  DRAW_STRING(70,130,'Darstellung der Mandelbrot-Menge. Hierzu müssen in F2 fünf');
  DRAW_STRING(70,150,'Eingabegrößen festgelegt werden. Bitte beachten Sie bei der');
  DRAW_STRING(70,170,'Eingabe,daß Sie keine Buchstaben eingeben. Verwenden Sie ');
  DRAW_STRING(70,190,'für Kommazahlen einen Punkt. Bei einer falschen Eingabe kann');
  DRAW_STRING(70,210,'durchaus vorkommen,daß das Programm abstürzt. Dieses schwer-');
  DRAW_STRING(70,230,'wiegende Problem konnte nicht gelöst werden, da es sich um');
  DRAW_STRING(70,250,'eine aufwendige Routine handelt ( Die Eingabe müßte als String');
  DRAW_STRING(70,270,'deklariert werden und auf jede Stelle untersucht werden ).');
  DRAW_STRING(70,290,'Dieses Programm wurde in Pascal erstellt. Pascal ist im Gegen-');
  DRAW_STRING(70,310,'satz zu Assembler relativ langsam. Dennoch wünsche ich dem ');
  DRAW_STRING(70,330,'Benutzer beim Erstellen von Fractale viel Spaß. (TASTE !)');
  I:=SCAN;
 END;

{------------------------------------------------------------------------
 HAUPTMENÜ
 ------------------------------------------------------------------------}
PROCEDURE HAUPTMENU;

 VAR A : INTEGER;

 BEGIN

  { Gestaltung des Hintegrunds }

  CLEAR_SCREEN;
  PAINT_STYLE(0);
  PAINT_RECT(1,1,638,398);
  PAINT_STYLE(1);
  PAINT_RECT(4,4,632,392);
  PAINT_STYLE(5);
  PAINT_RECT(5,5,630,390);

  { Fenster in der Mitte }

  PAINT_STYLE(0);
  PAINT_RECT(182,20,280,360);
  PAINT_RECT(184,22,276,356);
  PAINT_STYLE(4);
  PAINT_RECT(185,23,274,354);

  { Hintergrund der Menübox }

```

```
  PAINT_STYLE(0);
  PAINT_RECT(190,28,264,344);
  PAINT_RECT(192,30,260,340);
  PAINT_RECT(193,31,258,338);
  PAINT_STYLE(5);
  PAINT_RECT(194,32,256,336);

  { Beschriftung der Menübox }

  PAINT_STYLE(0);
  PAINT_RECT(199,40,245,55);
  PAINT_RECT(201,42,241,51);
  TEXT_STYLE(1);
  TEXT_HEIGTH(13);
  DRAW_STRING(250,65,'MANDELBROT - MENGEN');
  TEXT_HEIGTH(4);
  DRAW_STRING(220,78,'Das Computerprogramm zur Facharbeit');
  DRAW_STRING(220,88,'         von Urban Bettag          ');
  FOR A:=1 TO 6 DO
   BEGIN
    PAINT_STYLE(0);
    PAINT_RECT(199,79+(A*32),50,29);PAINT_RECT(279,79+(A*32),146,29);
    PAINT_STYLE(1);
    PAINT_RECT(204,84+(A*32),42,21);PAINT_RECT(284,84+(A*32),138,21);
    PAINT_STYLE(0);
    PAINT_RECT(202,82+(A*32),42,21);PAINT_RECT(282,82+(A*32),138,21);
   END;
  TEXT_STYLE(0);
  TEXT_HEIGTH(13);
  DRAW_STRING(212,130,'F 1');TEXT_HEIGTH(4);DRAW_STRING(288,122,'   Erstelle Bilder  ');
                                            DRAW_STRING(292,130,'Im- bzw. Exportieren');
  TEXT_HEIGTH(13);
  DRAW_STRING(212,162,'F 2');TEXT_HEIGTH(4);DRAW_STRING(292,158,' Fractale berechnen ');
  TEXT_HEIGTH(13);
  DRAW_STRING(212,194,'F 3');TEXT_HEIGTH(4);DRAW_STRING(292,186,' Aktuelle Parameter ');
                                            DRAW_STRING(292,193,'     einblenden     ');
  TEXT_HEIGTH(13);
  DRAW_STRING(212,226,'F 4');TEXT_HEIGTH(4);DRAW_STRING(292,222,' Fractal ausdrucken ');
  TEXT_HEIGTH(13);
  DRAW_STRING(212,258,'F 5');TEXT_HEIGTH(4);DRAW_STRING(274,294,'  Information für   ');
                                            DRAW_STRING(292,258,'     Benutzer       ');
  TEXT_HEIGTH(13);
  DRAW_STRING(212,290,'F10');TEXT_HEIGTH(4);DRAW_STRING(292,286,'  Programm beenden  ');
  TEXT_HEIGTH(13);
 END;

{------------------------------------------------------------------------
 BEENDEN
 ------------------------------------------------------------------------}

 PROCEDURE BEENDEN;

 BEGIN
  CLEAR_SCREEN;
  PAINT_STYLE(0);
  PAINT_RECT(1,1,638,398);
  PAINT_STYLE(1);
  PAINT_RECT(4,4,632,392);
  PAINT_STYLE(5);
  PAINT_RECT(5,5,630,390);
  PAINT_STYLE(1);
  PAINT_RECT(210,180,220,40);
  PAINT_STYLE(0);
  PAINT_RECT(208,178,220,40);
  DRAW_STRING(220,200,'Sind Sie sicher (J/N) ?');
  I:=SCAN;
  IF I=36 THEN EN:=FALSE;
```

```
  IF I=49 THEN EN:=TRUE;
 END;

{------------------------------------------------------------------------
 HAUPTPROGRAMM
 ------------------------------------------------------------------------}

BEGIN

 { GEM - Routinen werden aufgerufen und instaliert

   Init_Gem
   --------

   Die Funktion führt alle notwendigen Operationen durch, um GEM mitzuteilen
   daß nun eine Applikation an den Segnungen des Betriebssystems teilhaben
   möchte. Tritt ein Fehler auf so liefert die Funktion Init_gem der Wert -1
   Eine erfolgreiche Operation liefert immer einen positiven Wert.

   Exit_Gem
   --------

   Beendet ein GEM - Programm (Graphic Environment Manager). Die Prozedur
   teil GEM mit, daß ab nun keine weiteren GEM-Aufrufe getätigt werden.
   Bei einem fehlerhaften Aufruf von Init_Gem sollte man diese Prozedur nicht
   aufrufen. }

 IF INIT_GEM >=0 THEN
  BEGIN
   BILD_SPEICHER:=FALSE;
   EN:=TRUE;
   REPEAT
    HAUPTMENU;
    I:=SCAN;
    IF I=59 THEN IM_EXPORT;
    IF I=60 THEN BERECHNEN;
    IF I=61 THEN PARAMETER;
    IF I=62 THEN DRUCKEN;
    IF I=63 THEN INFO;
    IF I=68 THEN BEENDEN;
    SHOW_MOUSE;
   UNTIL EN=FALSE;
   EXIT_GEM;
  END;
END.
```

Notes on the program

- **Historical findings:** The program was evidently written for an Atari ST computer with the GEM interface and BIOS/XBIOS calls. Identifiers such as `TEXT_HEIGTH` have not been modernised, because they may come from the missing libraries.
- **Scan code:** The return value of `BCONCIN` contains the scan code in the upper word. This would call for a division by $65536 = 2^{16}$ or a shift by 16 bits. The printout reads `DIV (65535)`; this is very probably an error in the original program and has been deliberately retained.
- **Menu label:** In the main menu, the label “Information für” is at $y = 294$, and the corresponding second line “Benutzer” at $y = 258$. The two words therefore appear separately on the screen; this too is an error in the original and is preserved.

- **Grid bounds:** The loops each run from 0 up to and including 400, that is, over 401 values. At the same time, the real and imaginary parts are increased by one step before the first pixel. As a result, the stated minima are skipped and values above the maxima are reached at the end. In addition, the y-coordinate 400 may lie outside the intended grid for a screen area from 0 to 399.
- **Iteration:** For each parameter c, the computation in the program starts with $z = c$, not with $z_0 = 0$. For the pure escape test this amounts to a count shifted by one step; for the black-and-white colouring used, however, it changes the parity of the iteration count. Moreover, the termination sum `REAL_Q+IMAG_Q` is formed from the squares before the update just computed.
- **Representation:** Exterior points are drawn black or white according to the parity of the remaining counter. The stripes therefore show escape-time classes. They are not in themselves proof that a point belongs to the Mandelbrot set.
- **Present-day assessment:** With the missing GEM/TRIX files and a suitable historical Pascal system, the syntax could be checked more precisely. Without these components, any supposedly runnable reconstruction would be speculative; for this reason only the program text actually available has been included here.

5.5 Python post-processing from 2026

The following Python programs are not part of the original Facharbeit of 1990. They were written in 2026 for the digital revision. The first program computes the new black-and-white, colour and detail images of the Mandelbrot set. The second program produces the three-dimensional escape-time representation. Both programs use the mathematical iteration $z_{n+1} = z_n^2 + c$ with the starting value $z_0 = 0$.

In each case only the computational core is printed; the line numbers correspond to the complete file. The remaining parts of the programs merely produce labels and the image layout. The complete scripts `generate_fractals.py`, `generate_3d_fractal.py`, `generate_diagrams.py` and `generate_ifs.py` are not printed.

Escape times for the two-dimensional images

`generate_fractals.py`: function `escape_times`

```
40 def escape_times(bounds, iterations, width=1000):
41     """Berechne die erste Fluchtiteration fuer z_0=0, z_(n+1)=z_n^2+c."""
42     xmin, xmax, ymin, ymax = bounds
43     height = max(2, round(width * (ymax - ymin) / (xmax - xmin)))
44     x = np.linspace(xmin, xmax, width, dtype=np.float64)
45     y = np.linspace(ymin, ymax, height, dtype=np.float64)
46     c = x[np.newaxis, :] + 1j * y[:, np.newaxis]
47     z = np.zeros_like(c)
48     active = np.ones(c.shape, dtype=bool)
49     escaped = np.full(c.shape, iterations, dtype=np.uint16)
50 
51     for n in range(1, iterations + 1):
52         z[active] = z[active] * z[active] + c[active]
53         newly = active & (z.real * z.real + z.imag * z.imag > 4.0)
54         escaped[newly] = n
55         active[newly] = False
56         if not active.any():
57             break
58     return escaped
```

For each pixel c, the sequence is iterated from $z_0 = 0$ until $|z_n|^2 > 4$ holds or the iteration limit is reached. What is stored is the index of the first crossing, that is, the escape time.

Height field for the three-dimensional representation

`generate_3d_fractal.py`: function `height_field`

```
44 def height_field(nx=230, ny=190):
45     xmin, xmax, ymin, ymax = BOUNDS
46     xs = np.linspace(xmin, xmax, nx)
47     ys = np.linspace(ymin, ymax, ny)
48     c = xs[np.newaxis, :] + 1j * ys[:, np.newaxis]
49     z = np.zeros_like(c)
50     active = np.ones(c.shape, dtype=bool)
51     smooth = np.full(c.shape, float(MAX_ITER), dtype=np.float64)
52 
53     for iteration in range(1, MAX_ITER + 1):
54         z[active] = z[active] * z[active] + c[active]
55         radius2 = z.real * z.real + z.imag * z.imag
56         escaped = active & (radius2 > 4.0)
```

```
        if escaped.any():
            modulus = np.sqrt(radius2[escaped])
            smooth[escaped] = (
                iteration + 1.0 - np.log(np.log(modulus)) / np.log(2.0)
            )
        active[escaped] = False
        if not active.any():
            break

    # Schnelles Entweichen ergibt grosse Hoehen. Nicht entwichene Punkte sind
    # der ebene See auf Hoehe null. Die Wurzel macht die Kueste besser sichtbar.
    height = np.zeros_like(smooth)
    outside = ~active
    speed = np.clip((MAX_ITER - smooth[outside]) / MAX_ITER, 0.0, 1.0)
    height[outside] = 0.20 + 1.85 * np.sqrt(speed)
    return xs, ys, height, active
```

Here the escape time is smoothed and converted into a height: the faster a point escapes, the higher it lies. Points that have not escaped form the flat surface of the “Mandelbrot lake”.

Change log

The table lists the most important corrections of content on original pages 20 to 24. It is not exhaustive.

Page	Original problem	Correction	Effect
20	Starting value and index shifted	$z_0 = 0$, $z_1 = k$, $z_2 = k^2 + k$	Recursion and program count agree
21	Parameter k drops out	Case distinction explicitly restricted to $k = 0$	The existing section is kept as a special case
22	$\mathcal{M}$ defined via convergence to zero	Boundedness of the critical orbit	Correct definition of the set
22–23	Finite test taken as proof of membership	Non-escape up to N as an approximation	The algorithmic statement becomes precise
23	New components not written back	Update a and b after every step	The pseudocode actually performs an iteration
24	Parameter plane and starting-value plane conflated	Separate the Mandelbrot and Julia planes	The relationship between the two sets is described correctly

Page numbers refer to the 1990 original.

Source basis

The basis is the original scans `IMG_1917` to `IMG_1960`, covering printed pages 1 to 44. File number `IMG_1950` is missing; according to the author, the corresponding gap is closed by the colour plate `IMG_1960`, photographed later. For the partial check of the program, the visible program core in `IMG_1965` was also used. The historical corrections use mathematical foundations that were already available in 1990; later findings are explicitly marked as such.